



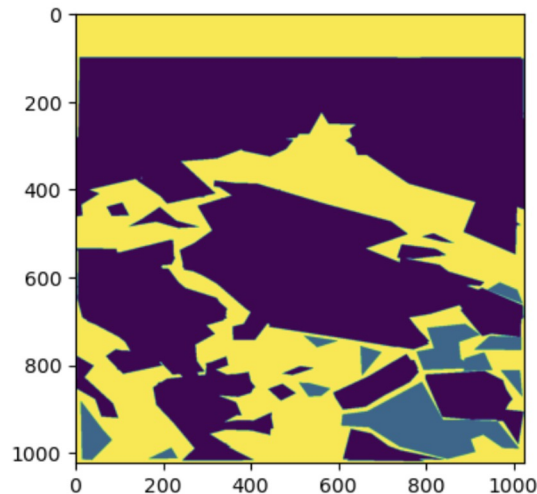
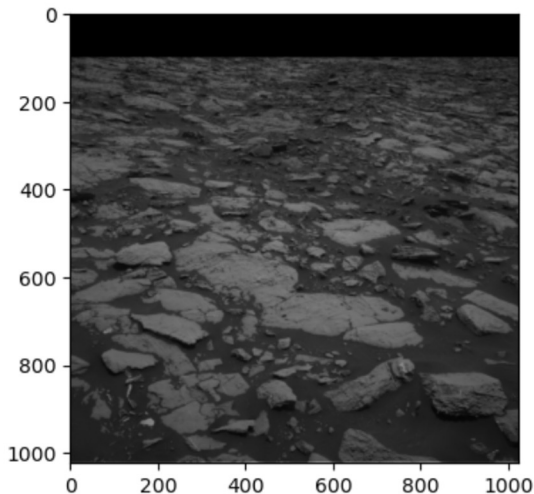
Mars Terrain Segmentation

Tzu-Han Lin, Can Xu, Hashir Mohyuddin

Intro of Our Dataset

There are two datasets in AI4MARS. Mars Exploration Rovers (MER) which were, collected by Spirit and Opportunity, and Mars Science Laboratory (MSL), collected by curiosity. For our research, we used only MSL dataset.

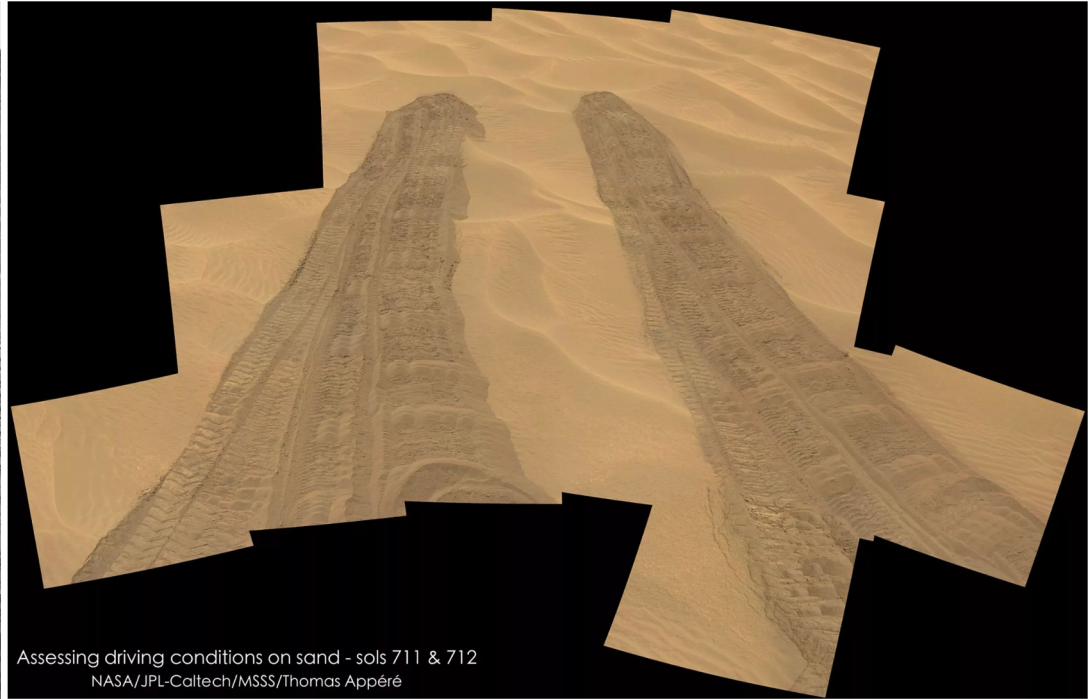
Label	Terrain Type
0	soil
1	bedrock
2	sand
3	big rock
255	background



Training Set: 16,064 images | Test Set: 322 images | Image Size: 1024 x 1024

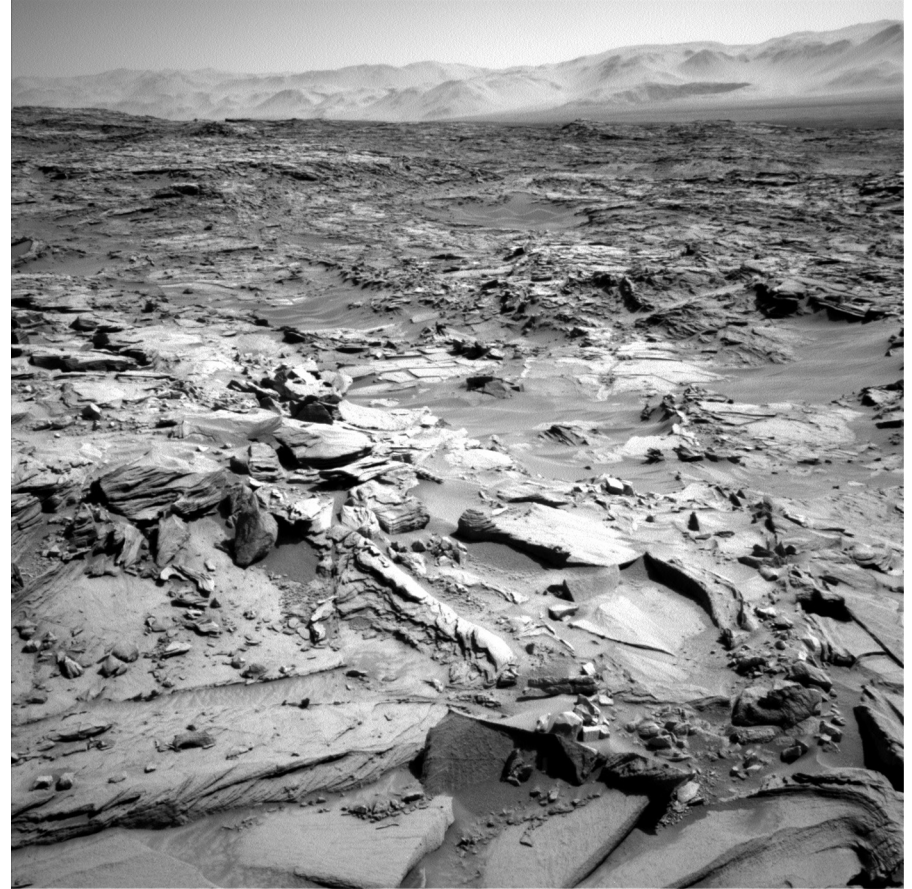
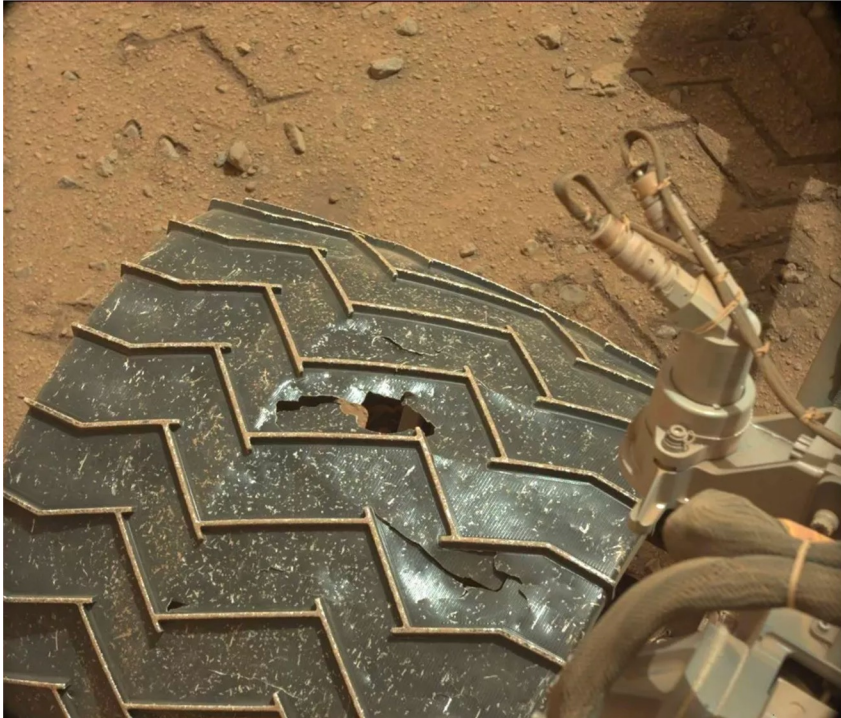
Problems Encountered

Photo taken by Curiosity on Sol 717. The narrow valley has a relatively steep slope on both sides and a floor that is constituted by ripple sand. The deep sand caused wheel slip and sinkage.



Problems Encountered

The accumulation of damage was caused by continuously driving on angular, embedded rocks



The Perseverance rover uses only a perception and navigation system based solely on classic machine vision algorithms. This means that auto-navigation evaluates the traversability of terrain using only geometric information, without taking into account semantic information such as terrain type.



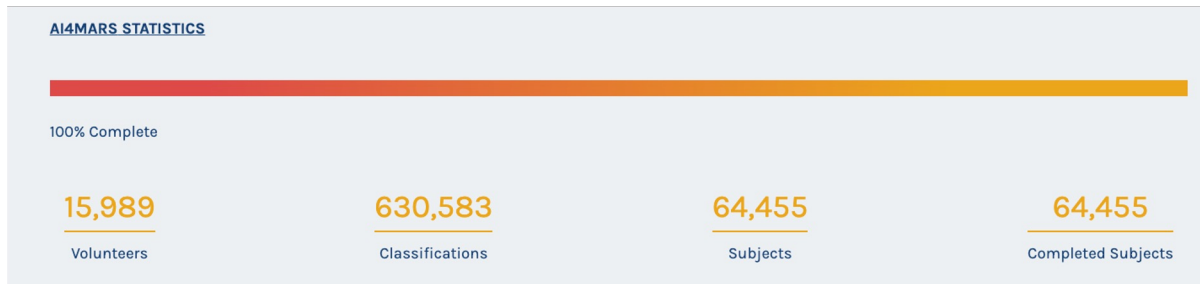
How did they get label information for such huge dataset?

Open the dataset for public contribution

<https://www.zooniverse.org/projects/hiro-ono/ai4mars/about/research>

For MSL and MER labels, a merged label requires three ‘citizen scientists’ to label a pixel with the same class and to have >65% agreement for each pixel.

We use the loose label policy where a pixel is labeled with a certain class if at least one ‘citizen scientist’ assigns it to that class



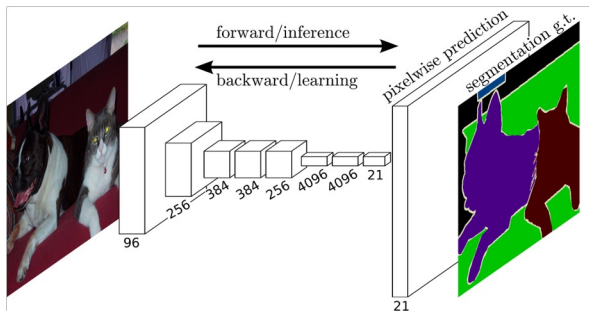
Masks the rover

In all images and any distance greater than 30 meters away.

```
if mxy_n in mxy:
    mask = np.array(cv2.cvtColor(cv2.imread(mxy_n), cv2.COLOR_BGR2GRAY))
    img[mask == 1] = 0
if rng_n in rng:
    mask2 = np.array(cv2.cvtColor(cv2.imread(rng_n), cv2.COLOR_BGR2GRAY))
    img[mask2 == 1] = 0
```



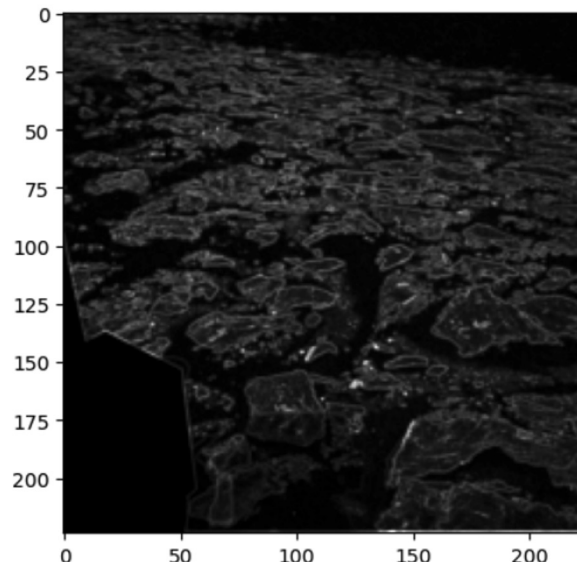
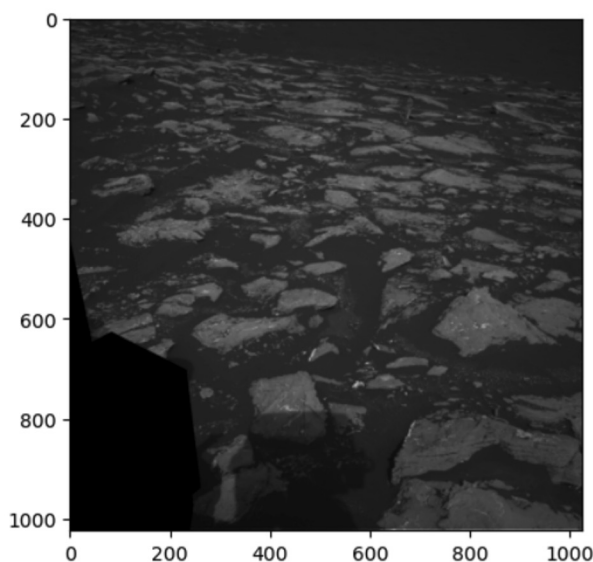
Method: FCN with VGG16 Backbone



```
dx = np.array([[[-1, 0, 1],  
                [-2, 0, 2],  
                [-1, 0, 1]])  
  
dy = np.array([[[-1, -2, -1],  
                [0, 0, 0],  
                [1, 2, 1]])
```

Innovation idea of this project:

Instead of three RGB channel, we add one channel to represent the edge in our model.



Evaluation method

Pixel accuracy, Precision, Recall

In cases where either the true positive count, false positive count, or false negative count is zero for a particular class during evaluation, the recall (or precision) metric may become undefined. To handle this, the ***np.nanmean*** function from NumPy can be used to calculate the average recall (or precision) across multiple classes while ignoring any NaN (not a number) values that may arise from 0/0 cases.

```
def result(l):  
    recall=[]  
    precision=[]  
    for i in range(len(l[0])):  
        avg_recall = np.nanmean(l[0][i])  
        recall.append(avg_recall)  
        avg_precision = np.nanmean(l[1][i])  
        precision.append(avg_precision)
```

$$PA = \frac{\sum_{i=0}^4 P_{ii}}{\sum_{i=0}^4 \sum_{j=0}^4 P_{ij}}$$

$$precision_i = \frac{P_{ii}}{\sum_{j=0}^4 P_{ij}}$$

$$recall_i = \frac{P_{ii}}{\sum_{j=0}^4 P_{ji}}$$

Experiment 1: With or Without Pretrained and Edge

Optimizer: Adam Loss function – CrossEntropyLoss

Name	Hyperparameter (batch, lr, epoch, pretrained)	[soil, bedrock, sand, bigrock, background] recall = r, precision = p, pixel accuracy=pa
msl_FCN_0420_03.pt	(16, 0.001,80, True) #didn't fix linear resize	r=[0.55, 0.61, 0.44, 0.07, 0.85] p=[0.81, 0.41, 0.30, 0.01, 0.83] pa= 0.84
msl_FCN_0420_pretrain edFalse.pt	(16, 0.001,80, False) #didn't fix linear resize	r=[0.56, 0.60, 0.44, 0.07, 0.86] precision [0.79, 0.41, 0.33, 0.01, 0.83] pa= 0.84
msl_FCN_0420_edge.pt	(16, 0.001,80, False) #didn't fix linear resize	r=[0.50, 0.64, 0.43, 0.07, 0.85] p=[0.83, 0.40, 0.32, 0.01, 0.81] pa= 0.83

With pretrained weights or not, it didn't really affect our results.

With edge, the bedrock type recall increase

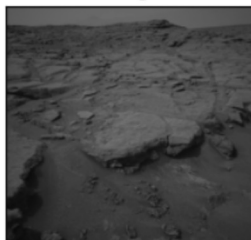
msl_FCN_0420_03.pt

msl_FCN_0420_pretrainedFalse.pt

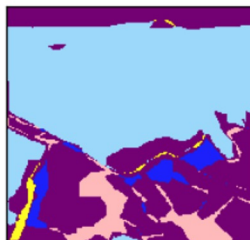
msl_FCN_0420_edge.pt

[soil, bedrock, sand, bigrock, background] = [softpink(#FFC2C2), light blue(#A2DBFA), blue, yellow, purple]

Image



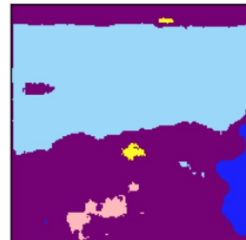
Ground Truth



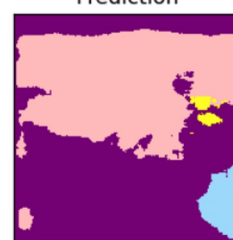
Prediction



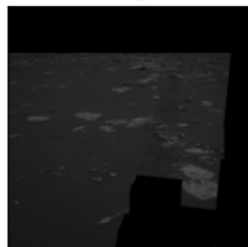
Prediction



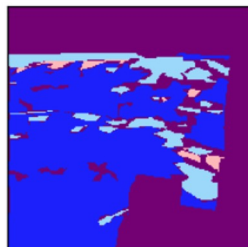
Prediction



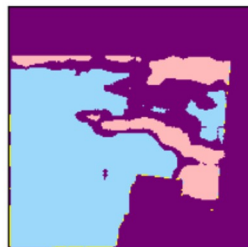
Image



Ground Truth



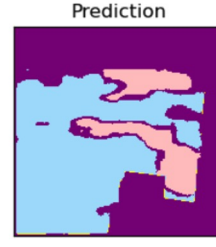
Prediction



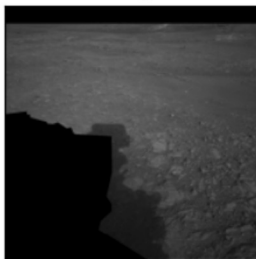
Prediction



Prediction



Image



Ground Truth



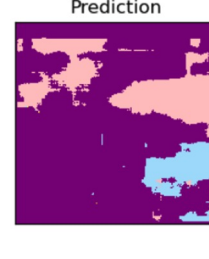
Prediction



Prediction



Prediction



Experiment 2: adjust hyperparameter and fix the linear resize problem

Name	Hyperparameter (batch, lr, epoch, pretrained)	[soil, bedrock, sand, bigrock, background] recall = r, precision = p, pixel accuracy=pa
mssl_FCN_0504_01.pt	(48, 0.0001, 100, True)	r= [0.59, 0.65, 0.44, 0.12, 0.85] p= [0.77, 0.67, 0.69, 0.15, 0.84] pa= 0.86
mssl_FCN_0502_01.pt	(48, 0.0001, 80, True)	r=[0.59, 0.65, 0.44, 0.08, 0.86] p=[0.79, 0.67, 0.76, 0.14, 0.84] pa = 0.86
mssl_FCN_0420_03.pt	(16, 0.001, 80, True) #didn't fix linear resize	r=[0.55, 0.61, 0.44, 0.07, 0.85] p=[0.81, 0.41, 0.30, 0.01, 0.83] pa= 0.84

Lower the learning rate, and increasing the batch size helps increase the recall and precision.

When epoch is increased to 100, the model starts overfitting; based on the val loss and train loss. Performance does not increase when epoch is greater than 80.

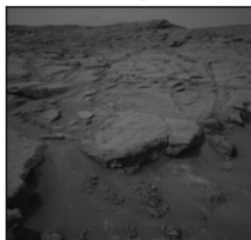
msl_FCN_0420_03.pt

msl_FCN_0504_01.pt

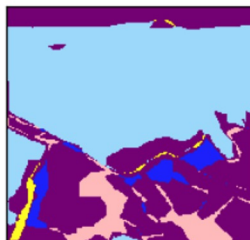
msl_FCN_0502_01.pt

[soil, bedrock, sand, bigrock, background] = [softpink(#FFC2C2), light blue(#A2DBFA), blue, yellow, purple]

Image



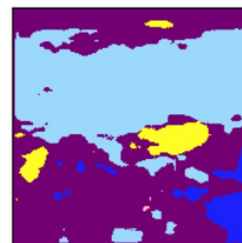
Ground Truth



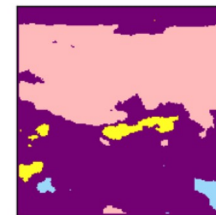
Prediction



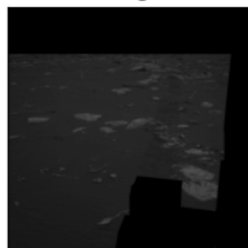
Prediction



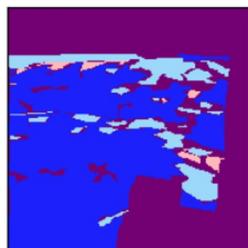
Prediction



Image



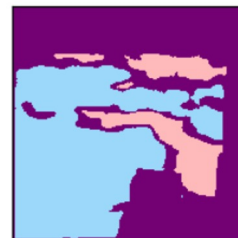
Ground Truth



Prediction



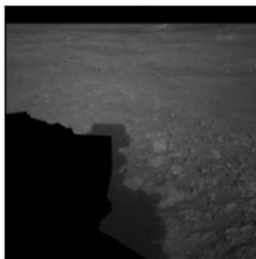
Prediction



Prediction



Image



Ground Truth



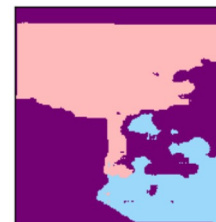
Prediction



Prediction



Prediction



Conclusion

We found that adding an additional edge channel helps to better identify bedrock terrain types. Utilizing this technique can aid to prevent potential damage to the rover's wheels.

However, our model's ability to incorporate new data is not very good. Future work can look at adjusting hyperparameters to improve performance (especially at sand type).

Additionally, in our dataset, big rock types are scarce leading to low performance for that specific feature. Incorporating data augmentation techniques during training to increase the diversity of the data can help to circumvent this issue.

We also discovered that our labels may not be completely accurate.

